NPM3D Project Report - MVA 2025/2026
Point Transformer (Hengshuang Zhao et al., 2020)

Alexandre Devaux-Rivière
alexandre.devaux--riviere@ens-paris-saclay.fr

Abstract

In this report, I propose a study of the Point Transformer [17], a transformer-based architecture for 3D point cloud understanding. The paper introduces a specialized self-attention operator tailored to irregular point cloud processing, combining local neighborhood attention, relative positional encoding, and vector attention. The method achieves strong results on several benchmark tasks such as semantic scene segmentation, part segmentation, and shape classification.

I focus on reproducing the results of the original paper on the ModelNet40 [15] shape classification task. My implementation achieves an overall accuracy of 92.7%, close to the 93.7% reported by the authors. Beyond reproduction, I conduct a series of ablation studies and architectural modifications to better understand the behavior of the model.

1. Introduction

Point cloud processing has become a more and more popular topic in 3D deep learning because many modern applications naturally produce data in the form of point sets. To name a few, we can cite LiDAR perception for autonomous driving, robotics, augmented reality, videogames, medical imaging, and 3D scene understanding. However, the intrinsic nature of this data poses challenges for traditional deep learning architectures, as most existing models assume data organized on a regular grid with fixed neighborhood relationships. In contrast, point clouds are irregular unordered sets of points distributed in continuous 3D space, which complicates their direct processing with conventional methods.

To address this issue, several early families of approaches have been proposed. Some pointset processing methods attempt to force point clouds into structured representations, such as voxel grids [18] or multi-view images [8], in order to leverage existing architectures, like classical convolutional architectures (CNNs) that were the standard before the publication of this paper. However, applying these transformations to discretize the space of point sets inevitably leads to a loss of information about the geometric structure of the original data and suffers from high computational complexity due to the scarcity of 3D points. This motivated the shift towards point-based and graph-based neural networks that operate directly on raw point sets to preserve their original geometric structure.

More recently, transformer architectures and self-attention models have revolutionized deep learning in natural language processing and computer vision with Image Transformers [5] [10], Vision Transformers (ViT) [4], and self-attention-based image recognition [16]. Their core operator is particularly appropriate for point clouds because the self-attention operator is, in essence, a set operator: it is invariant to permutation and cardinality of the input elements. This observation directly motivated the Point Transformer paper. Building on the self-attention mechanism introduced in [16], Zhao et al. adapt the transformer principle to 3D point clouds by introducing a local self-attention layer with trainable relative position encoding and vector-valued attention weights.

When introduced, the Point Transformer achieved state-of-the art results on several benchmark datasets, including large-scale semantic segmentation on the S3DIS dataset [1] (70.4% mIoU on Area 5), shape classification on Model-Net40 (93.7% overall accuracy), and object part segmentation on ShapeNetPart [2] (86.6% instance mIoU).

For this project, I aimed to reproduce and study the results of the Point Transformer paper on the ModelNet40 shape classification dataset [15]. Based on the PyTorch training pipeline provided in the non-official GitHub repository, which exclusively focuses on segmentation, I conducted a detailed study to evaluate how different network modules and parameters contribute to final overall performance. Specifically, I went through the following steps:

- First, I adapted the existing Point Transformer code to transition the project from a segmentation task to a shape classification task.
- Second, I attempted to match the original paper's per-

formance by refining the model through iterative testing on the ModelNet40 dataset.

- Next, I tested the model's reliance on local geometry by mapping its sensitivity to the neighborhood size parameter k.
- Then, I dissected the core self-attention mechanism through ablation studies by isolating the roles of positional encoding and contrasting the vector attention with a scalar dot-product alternative and standard MLPs.
- Finally, I analyzed the training dynamics and evaluated the influence of different optimizers and learning rate schedulers on the model's final overall performance and convergence speed.

2. Related Work

2.1. Point Cloud Processing

The fundamental challenge in 3D point cloud understanding is their inherently unordered, scattered nature and varying cardinality. Learning-based approaches designed to handle this irregularity can broadly be classified into three categories: projection-based, voxel-based, and point-based networks.

Voxel-based approaches directly rely on 3D voxelization, allowing for the application of standard 3D CNNs. However, when applied naively, this induces enormous computational and memory costs due to the cubic growth of the number of voxels relative to resolution. Even though solutions exist to mitigate these limitations by operating only on occupied voxels, such as OctNet's unbalanced octrees or sparse convolution operators, these methods may still lose fine geometric details due to quantization onto the grid.

Projection-based / view-based methods offer an alternative by rendering 3D point clouds into regular 2D image planes from multiple viewpoints, enabling processing via standard 2D CNNs to extract features followed by multi-view feature fusion. Other related approaches, like Tangent-Conv, project local surface geometry onto tangent planes. While some of these methods yield excellent benchmark results, they suffer from geometric information loss during projection, are negatively impacted by 3D occlusion, and the choice of projection planes considerably affects performance.

More recently, several architectures have emerged that completely bypass grid quantization to learn powerful representations directly on 3D point clouds, thereby preserving their original geometric structure. Point-based architectures like PointNet [6] achieve this by utilizing permutation-invariant operators like pointwise MLPs and pooling layers. PointNet++ [7] applies these within a hierarchical spatial structure using sampling strategies. Other active research areas include graph-based approaches like DGCNN [12], which leverage graph convolutions on kNN graphs for message passing, and continuous convolution methods like KPConv [9], which operate within spherical neighborhoods using learnable kernel points in rigid / deformable layouts.

2.2. Attention and Transformers

The Transformer architecture, initially conceived as an evolution of sequence-to-sequence models for NLP, has since shifted the deep learning paradigm and advanced nearly every machine learning discipline by overcoming the structural limitations of earlier models. While recurrent neural networks (RNNs) struggle to capture long-term dependencies and CNNs are restricted by an inherently local focus that misses distant correlations, Transformers solve this by utilizing attention mechanisms to examine an entire data sequence simultaneously. At the core of this revolution is self-attention, which allows the model to intricately discern how every part of an input influences and correlates with the others, deciding exactly how and when to focus on specific elements regardless of their distance.

This mechanism operates by generating query (Q), key (K), and value (V) vectors for each element in the input. The query vector represents the information a specific element is seeking, while the key vector represents the information it contains. To determine the relative importance between elements, the model computes the dot product between their query and key vectors. These alignment scores are then passed through a softmax activation function, converting them into normalized attention weights. The value vector returns the information from the key, scaled by this computed attention weight. This core operation, which is typically expanded into multi-head attention to capture multifaceted relationships (see Figure 1), is formally defined as:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad (1)$$

Because the self-attention operation processes elements simultaneously rather than sequentially, it is permutation-invariant. However, because the order and position of elements often carry important semantic meaning, transformer models inject positional information into the attention mechanism. This is done through positional encoding, where a vector derived from an element's relative position is added to the initial embedding before it is processed by the attention layers. Consequently, its properties make self-attention theoretically ideal for 3D point clouds, bypassing the need to artificially construct a sequence and instead relying directly on 3D spatial coordinates to inform the model of the local geometry.

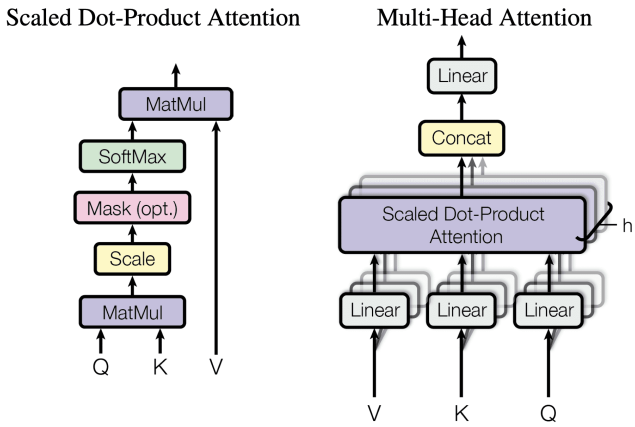

Figure 1. Scaled dot-product attention (left). Multi-Head Attention consists of several attention layers running in parallel (right). (Source: [11])

3. Presentation of the Point Transformer Method

3.1. Point Transformer Layer

The core contribution of the paper is the Point Transformer layer. Its role is to take as input a set of 3D points represented by their coordinates and associated features and to produce updated features for the same points. As in the transformer family, the output representation of each point is obtained by aggregating information from neighboring points through a custom attention mechanism. As mentioned above, this attention mechanism is not exactly the same as Equation (1) and we can rewrite it as:

$$\mathbf{y}_i = \sum_{\mathbf{x}_j \in X} \rho\left(\phi(\mathbf{x}_i)^T \psi(\mathbf{x}_j) + \delta_{i,j}\right) \alpha(\mathbf{x}_j) \qquad (2)$$

where $X = \{\mathbf{x}_i\}_i$ is the set of input features and $\mathbf{y}_i$ is the i-th output feature. Here, we can recognize ϕ, ψ, and α respectively playing the role of *query*, *key*, and *value* in Equation (2), to which a relative positional encoding δ is added. Specifically $\delta_{i,j}$ is defined as $\delta_{i,j} = \theta(\mathbf{p}_i - \mathbf{p}_j)$, where $\mathbf{p}_i$ and $\mathbf{p}_j$ denote the 3D coordinates of the respective points. This term is computed from the relative displacement through a small learnable multilayer perceptron θ, allowing the attention layer to incorporate geometric information and adapt to local structures. This positional encoding is important because point clouds are embedded in 3D space, where the relationships between points are fundamentally dependent on their spatial proximity and orientation. In this context, ρ is a normalization function, usually $\mathrm{softmax}$, which produces a probability distribution with coefficients in $[0, 1]$ for scalar attention.

However, instead of computing standard scalar dot-product attention, the paper proposes the intuition of using vector attention [16] for this layer. More precisely, the attention weight between a point and one of its neighbors becomes a vector itself, allowing each feature channel to be weighted differently and thus enabling the modulation of the input features on a finer scale. Furthermore, since the attention mechanism acts on sets, the authors optimize it by applying attention locally. Only the k-nearest neighbors of $\mathbf{x}_i$ in the 3D space are considered, substituting the global set X with a local subset $\mathcal{X}(i)$. In this way, the equation can be rewritten as follows, describing what they called vector attention:

$$\mathbf{y}_i = \sum_{\mathbf{x}_j \in \mathcal{X}(i)} \rho\big(\gamma\big(\phi(\mathbf{x}_i) - \psi(\mathbf{x}_j) + \delta_{i,j}\big)\big) \odot \big(\alpha(\mathbf{x}_j) + \delta_{i,j}\big)$$

where $\odot$ denotes the Hadamard product of the two vectors. In this formulation, the mappings ϕ, ψ, and α are linear transformations, while γ (the mapping function producing attention vectors) and θ (producing δ) are MLPs. Since all components are differentiable, everything can be learned end-to-end through backpropagation, and the attention mechanism is then illustrated by the following diagram:

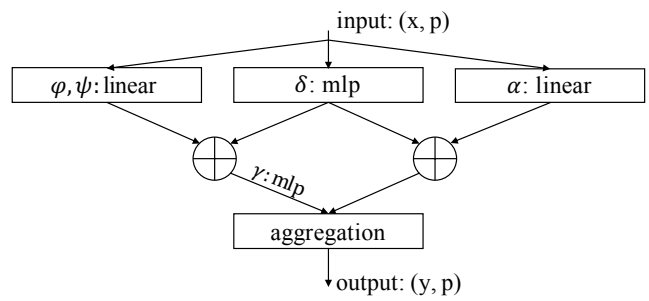

Figure 2. Schematic of the Point Transformer layer (source: [17])

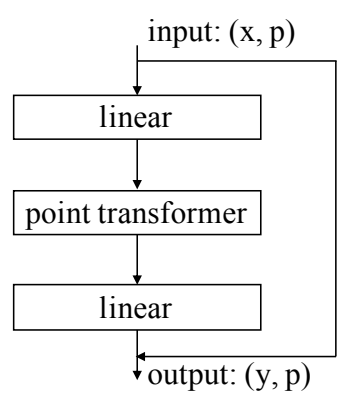

Figure 3. Schematic of the Point Transformer block (source: [17]).

Ultimately, this design makes the layer permutation-invariant and suited for unordered point sets. To build a practical network, the Point Transformer layer serves as the

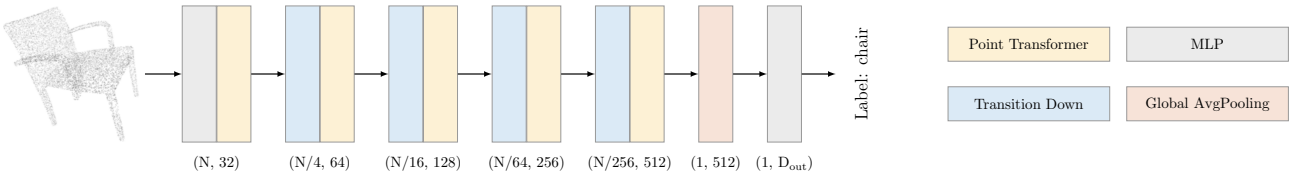

Figure 4. Architecture of the Point Transformer network for shape classification (source: [17]).

core of a larger residual block. Alongside the attention mechanism and nonlinearities, this block incorporates linear projections to lower the computational cost of propagation and backpropagation. Similar to ResNet architectures, this unit incorporates a skip connection to stabilize optimization and facilitate the learning of deeper hierarchical models. The resulting Point Transformer block is schematized in Figure 3:

3.2. Network Architecture for Shape Classification

The full Point Transformer network constructs a complete 3D point cloud understanding framework based entirely on Point Transformer layers, pointwise transformations, and pooling operations, without relying on convolutions for preprocessing or auxiliary branches. While the original framework proposes a hierarchical encoder-decoder architecture designed for dense prediction tasks such as semantic segmentation, my work focuses exclusively on 3D shape classification. Consequently, I adopt only the encoder component of this architecture, omitting the decoder, as the latter is not required for classification tasks. This encoder-only design is illustrated in Figure 4.

The backbone of the feature encoder consists of the succession of five Point Transformer and Transition Down blocks operating on progressively downsampled point sets. First, the network accepts an input point cloud of size N, utilizing the 3D coordinates (and normals, if available) as initial pointwise features. An initial MLP transforms these features into a vector of size 32, which is then processed by the first Point Transformer block. Consecutive stages in the network are connected by transition modules. These Transition Down layers successively reduce the cardinality of the point set by a factor of four, giving progressively smaller point sets of size N, $N/4$, $N/16$, $N/64$, and $N/256$, while simultaneously increasing the feature dimensionality. After the final stage, the encoder has extracted high-level, hierarchical features. Because the decoder is not needed for this task, a global average pooling layer is applied directly over the final pointwise features to obtain a 512-dimensional global shape descriptor for the entire point set. Finally, this global feature vector is passed through an MLP to generate the global classification logits.

3.3. Transition Down layer

Its primary role is to reduce the cardinality of the point set and aggregate local features from a denser set to a smaller set of representative points, decreasing the computational complexity of the network in deeper stages.

Suppose P_1 is the input set of points provided to the module. The goal of this layer is to retrieve a well-spread new subset $P_2 \subset P_1$, while computing accurate, aggregated features for the points in P_2. To achieve this, the operation proceeds through the following stages:

Sampling. First, Farthest Point Sampling is applied to P_1 to extract the subset P_2 at the required downsampling rate (for example, from N to $N/4$ in the transition from the first to the second stage).

Grouping. To pool feature vectors from P_1 onto P_2, we retrieve its k nearest neighbors in the set P_1 (using $k = 16$ throughout).

Transformation and Pooling. Each extracted input feature from the neighborhood goes through a pointwise linear transformation, followed by batch normalization and a ReLU activation. Finally, these features are max-pooled from the k neighbors in P_1 onto each point in P_2 to obtain the updated feature vector.

The resulting block is schematized in Figure 5:

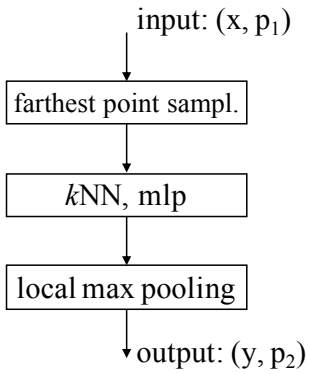

Figure 5. Schematic of the Transition Down block (source: [17]).

It is worth noting that for dense prediction tasks such as point cloud segmentation, a corresponding Transition Up layer is used to decode and increase the cardinality of the point set. However, since my implementation focused strictly on shape classification, this decoding layer was not required. Interested readers can refer to Section 3.5 of the original paper for further details on feature decoding.

4. Implementation Details and Methodology

4.1. From Segmentation to Classification

Due to the absence of official code attached to the paper, I started from the provided public unofficial POSTECH-CVLab Point Transformer repository. However, this implementation was designed for segmentation tasks and did not directly provide a classification pipeline for ModelNet40. Therefore, an important part of my work consisted in adapting the codebase to support shape classification. I introduced a dedicated classification architecture, and tried to keep it as close as possible to the encoder of the original paper. I also added support for ModelNet40 through a dataset loading class specifically designed for this benchmark. In addition, I modified the training script so that the model, dataset, optimizer, scheduler, and experimental parameters could be selected through configuration files and command-line options, which is necessary to perform controlled ablation studies.

4.2. Experimental Setup and Data Pre-Processing

All experiments were conducted on my personal laptop equipped with an NVIDIA RTX 3070 GPU. Because of this hardware constraint, the experiments were designed to remain feasible in terms of memory usage and training time.

The classification task is performed on ModelNet40 [15], a standard benchmark for 3D object classification used to evaluate point-based learning methods. The dataset contains 12,311 CAD models with 40 object categories. They are split into 9,843 models for training and 2,468 for testing. In the paper, Point Transformer reports strong performance on this dataset, which makes it a natural benchmark for reproducing and extending the classification setting. Since the original ModelNet40 dataset consists of CAD models, one must first extract point clouds from them. For these experiments, I used the modelnet40_normal_resampled dataset (available on Kaggle and Hugging face), where each model has been pre-processed into a dense point cloud consisting of 10,000 points with normals computed from the original faces. Additionally, since the models are derived purely from geometry, the 3D points do not provide any color information.

To greatly reduce computational complexity during training, I used the SharedArray library in my implementation to cache these pre-processed arrays directly into shared memory. This eliminates the bottleneck of repetitive disk reads for every batch. Furthermore, rather than executing a computationally expensive Farthest Point Sampling (FPS) step at the data-loading stage, I configured the dataloader to dynamically retrieve a subset of 4,000 points using uniform random sampling on the fly. FPS is instead reserved for the network's internal Transition Down layers to progressively downsample the point clouds' spatial resolution.

4.3. Training Pipeline and Hyperparameters

Since the numerical results will be added later, this section focuses on the design of the experiments. For training, I used my modified pipeline based on PyTorch, with data loading, augmentation, optimization, validation, checkpointing, and TensorBoard logging. Furthermore, I integrated a profiler to accurately track the execution time for each individual component of the architecture (during the evaluation stages). The training code supports different optimizers with optional gradient clipping, different learning rate schedulers, several positional encoding variants, multiple attention formulations, and variable neighborhood sizes. Specific to the classification task, I applied a data augmentation pipeline consisting of random scaling (with a factor uniformly drawn between 0.85 and 1.15), random rotation, and random jittering with $\sigma = 0.01$.

Since this is a classification task, I used the cross-entropy loss for training. The base optimizer that is used in the original paper, and implemented in this setup, is the SGD optimizer with momentum set to 0.9 and weight-decay set to 0.0001. The initial learning-rate is 0.05 and is dropped by ten-fold at steps 60 and 80 when training for 100 epochs (managed via a MultiStepLR scheduler). Note that the original study trains for 200 epochs.

As mentioned in the previous section, many functions within the network are either linear transformations or MLPs. Here are the exact details of what I used for them, adapting the original paper's suggestions to this PyTorch architecture, while keeping the liberties taken in the original code:

- ϕ, ψ, α: `Linear(d, d)` (the weights are not shared between the 3 functions).
- δ (Positional encoding): `Linear(3, 3)` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(3, d)`.
- γ (Attention mapping): `BatchNorm1d` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(d, d//8)` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(d//8, d//8)`.
- Linear projection of the Point Transformer block: `Linear(d_feature, d)`.

- MLP of the Transition Down layer: `Linear(d_feature, 2d_feature)` → `BatchNorm1d` → `ReLU` (applied to grouped coordinates/features).

- First MLP of the full model (Initial Transition Down): `Linear(3, 32)` → `BatchNorm1d` → `ReLU`.

5. Ablation Studies and Results

In the following ablation studies, optimal hyperparameters are determined sequentially. Unless stated otherwise, once a specific configuration is shown to yield the best performance, it is permanently adopted for all subsequent experiments. Here, I follow the data preparation procedure of Qi et al. [7] and uniformly sample 4000 points from each CAD model together with the normal vectors from the object meshes. For evaluation metrics, we use the mean accuracy within each category (mAcc) and the overall accuracy (OA) over all classes.

5.1. Impact of Neighborhood Size

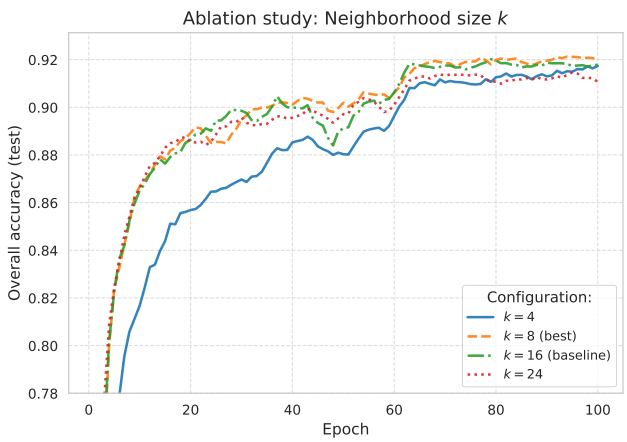

Figure 6. Ablation study: overall test accuracy over 100 epochs for different neighborhood size k (smoothed curves for clarity).

As previously discussed, the size of the local neighborhood k is an important hyperparameter dictating the receptive field of the local self-attention mechanism and the number of neighbors used for feature aggregation via local max pooling during point set downsampling. In the original paper, the authors proposed $k = 16$ as a baseline for their transition down modules. To investigate the sensitivity of the network to this parameter, I evaluated the model with $k \in \{4, 8, 16, 24\}$. The training dynamics are illustrated in Figure 6, and final results are shown in Table 1.

The best performance is achieved with $k = 8$ (92.67%), outperforming the baseline of $k = 16$ (92.18%) proposed by the authors. This superiority is confirmed by the $k = 8$ curve which maintains a slightly higher and stable accuracy trajectory compared to the other configurations in the second half of the training process. Furthermore, we observe that further increasing the neighborhood size to $k = 24$ degrades performance due to the introduction of excessive noise, while drastically increasing the total training time to 5.70 hours, which is more than double the time required for $k = 8$. Conversely, an overly small neighborhood like $k = 4$ restricts the contextual information available to the attention mechanism, slightly reducing accuracy. Therefore, $k = 8$ represents the optimal trade-off in terms of both computational efficiency and shape classification accuracy for the ModelNet40 dataset, as illustrated in Figure 7.

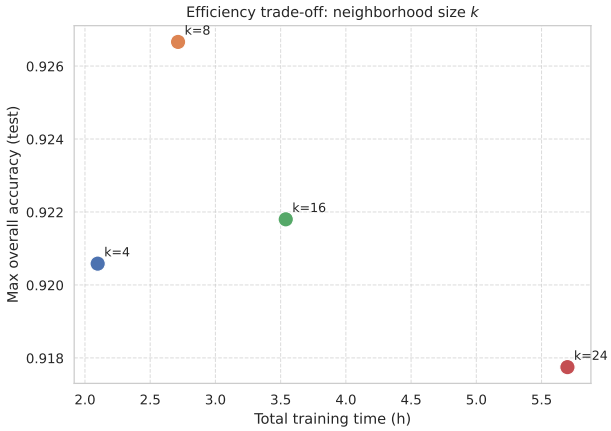

Figure 7. Efficiency trade-off: neighborhood size k

k	Max OA	Max mAll	Total time (h)
4	92.06	89.24	**2.10**
8	**92.67**	**90.25**	2.71
16	92.18	89.72	3.54
24	91.77	88.98	5.70

Table 1. Quantitative results for the neighborhood size (k) ablation study.

5.2. Impact of the Attention Mechanism

The main contribution of the Point Transformer architecture is its local vector attention mechanism. To validate its necessity, I compared this approach with my scalar dot-product attention implementation (see Equation (2)), as well as with simple MLPs, both with and without pooling (which serve as baselines).

In my experiments, completely replacing the attention layer with simple MLPs or with pooled MLPs failed to achieve convergence, producing results comparable to a random classifier (as shown in Table 2). However, since the authors of the original paper reported viable results using this simplified approach, it would be interesting to have access to their precise implementation details to properly

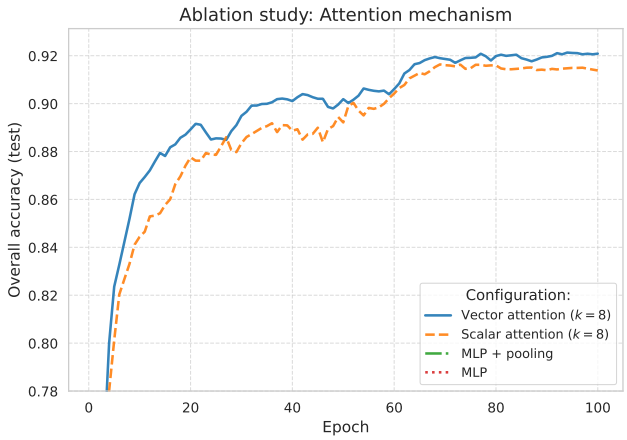

Figure 8. Ablation study: overall test accuracy over 100 epochs for different attention mechanisms (smoothed curves for clarity).

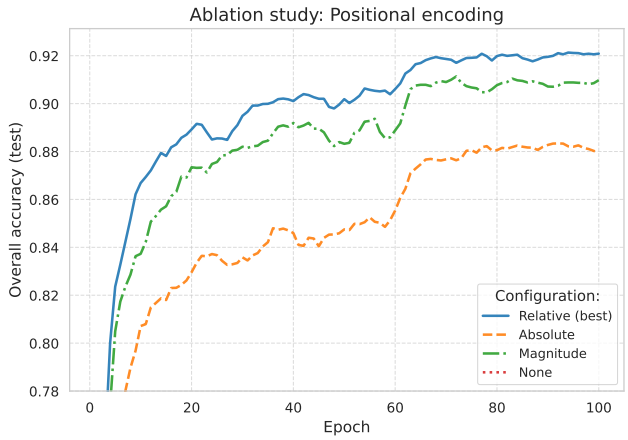

Figure 9. Ablation study: overall test accuracy over 100 epochs for different positional encoding (smoothed curves for clarity).

replicate this part of the study. I hypothesize that this gap stems from differences in datasets and downstream tasks: the original study used S3DIS, which includes color information, while ModelNet40 relies solely on structural geometry. Without the attention mechanism to dynamically model and aggregate local geometric relationships, the network cannot capture the defining spatial topologies of CAD models, leading to classification failure.

As expected, vector attention outperforms the scalar dot-product formulation. Furthermore, the learning curves in Figure 8 show that vector attention maintains a slight but consistent lead throughout the entire training process. Thus, although scalar attention is less computationally intensive, vector attention's capacity to modulate individual feature channels seems to offer an improved ability to capture 3D geometric structures, thereby justifying its higher computational cost. I hypothesize that this advantage would be even more noticeable on a more complex task or dataset (such as scene segmentation).

Attn. mechanism	Max OA	Max mAll	Time (h)
Vector attention	**92.67**	**90.25**	2.71
Scalar attention	92.18	89.57	**1.82**
MLP+pooling	4.05	2.50	0.89
MLP	4.05	2.50	0.86

Table 2. Quantitative results comparing vector attention and scalar dot-product attention.

5.3. Impact of Positional Encoding

Building upon the previous sections, the original architecture relies on relative positional encoding $\delta_{i,j} = \theta(\mathbf{p}_i - \mathbf{p}_j)$ to inject spatial information. To assess its impact, I tested four configurations: the baseline relative encoding, absolute coordinate encoding ($\delta_{i,j} = \theta(\mathbf{p}_i)$), magnitude-only encoding based on the element-wise absolute difference ($\delta_{i,j} = \theta(|\mathbf{p}_i - \mathbf{p}_j|)$), and no positional encoding at all. The final results are shown in Table 3.

The absence of positional encoding causes a catastrophic failure of the model, reducing its performance to that of a random classifier. Although the original paper also reported an important drop in performance without positional encoding for their dense prediction task, their model did not completely fail. Aside from this discrepancy, we can observe that the absolute and magnitude encodings perform considerably better. Absolute encoding provides global spatial awareness but lacks translation invariance, while magnitude encoding provides translation-invariant local distances but loses crucial directional context. Consequently, both approaches still fall short of the relative encoding strategy (92.67%), which maintains a consistent advantage throughout the entire training process as illustrated in Figure 9.

Pos. encoding	Max OA	Max mAll	Total time (h)
Relative	**92.67**	**90.25**	2.71
Absolute	88.98	85.07	**2.57**
Magnitude	91.57	88.75	2.73
None	4.05	2.50	1.86

Table 3. Quantitative results for the positional encoding strategy ablation.

5.4. Impact of Optimizers and Schedulers

Finally, I investigated the impact of the optimization strategy on the network's convergence. It is interesting to note that the authors propose using a standard SGD with a MultiStep scheduler, reducing the learning rate by a factor of 10 at 60% and 80% of the total number of epochs, whereas modern architectures like Vision Transformers often favor AdamW combined with a OneCycleLR scheduler

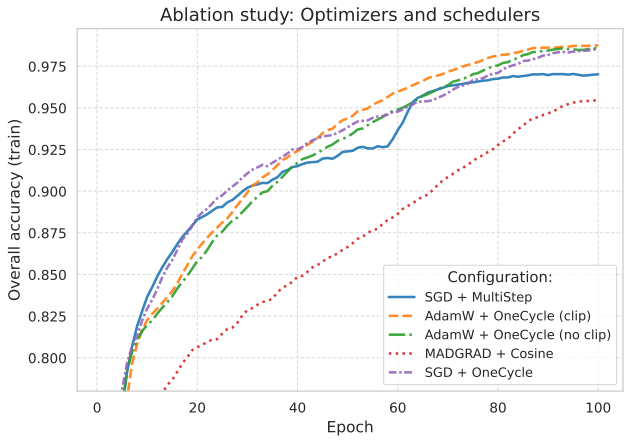

Figure 10. Ablation study: overall training accuracy over 100 epochs for different optimizers and schedulers (smoothed curves for clarity).

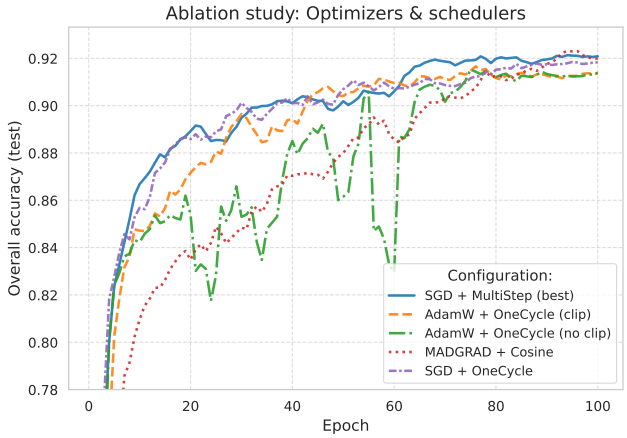

Figure 11. Ablation study: overall test accuracy over 100 epochs for different optimizers and schedulers (smoothed curves for clarity).

(for warmup) or Cosine Annealing.

As we can see in Table 4, the original setup remains the most effective with the best overall accuracy (92.67%), closely followed by the MADGRAD optimizer with Cosine Annealing. Both AdamW configurations reach the same peak overall accuracy but their training dynamics differ (see Figure 11). Without gradient clipping, AdamW is highly unstable, with large accuracy drops, whereas clipping stabilizes convergence. However, even when stabilized, AdamW slightly underperforms compared to SGD.

Comparing Figures 10 and 11, we observe a disparity in overfitting levels depending on the configuration. AdamW and SGD combined with a OneCycle scheduler achieve the highest final training overall accuracies and show a more pronounced generalization gap. The SGD+MultiStep configuration maintains a smaller margin between training and test performance. It is also worth noting that MADGRAD + Cosine Annealing has the greatest generalization capacity, exhibiting the smallest generalization gap among all the tested methods. Given the behavior of its corresponding curves, it is even likely that if training were to continue beyond 100 epochs, this configuration would ultimately achieve the best average overall accuracy.

Optim. & Sch.	Max OA	Max mAcc	T. Time (h)
SGD+Multi lr: 0.025, wd: 1e-4	**92.67**	90.25	**2.71**
SGD+1Cycle lr: 0.025, wd: 1e-4	92.30	**90.60**	2.73
AdamW (C) lr: 0.001, wd: 1e-2	92.06	89.85	3.09
AdamW (NC) lr: 0.002, wd: 2e-2	92.06	89.59	2.93
MadGrad lr: 0.005, wd: 1e-4	92.50	89.42	2.90

Table 4. Quantitative results for different optimizer and scheduler configurations. All experiments use a batch size of 16 over 100 epochs. (C): gradient clipping, (NC): no clipping.

6. Model Limitations

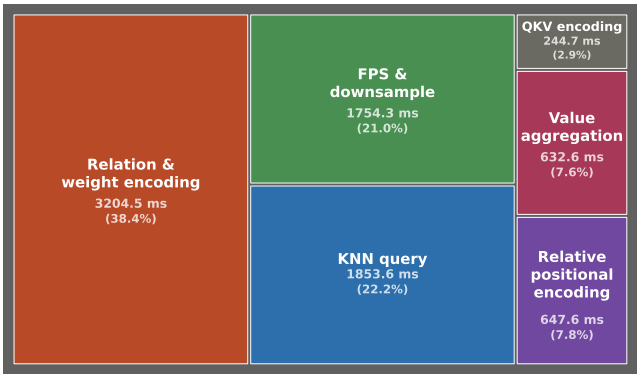

Figure 12. Benchmark: latency treemap of each components and visualization of the proportion of the forward time of each component of PTv1 during evaluation.

Although the original Point Transformer architecture is innovative for 3D point cloud understanding, it presents several important limitations not mentioned in the original paper that hinder its scalability and efficiency.

Indeed, as the network depth and channel dimensions increase, the MLPs used for weight encoding in its vector attention mechanism experience a sharp increase in the number of parameters, which ultimately leads to overfitting and limits the viable depth of the model. As the profiling figure shows, this relation and weight encoding step alone dominates the computational load, taking up 38.4% of the forward pass (see Figure 12).

Furthermore, its spatial pooling mechanism relies on a combination of farthest point sampling (FPS) and k-Nearest

Neighbor (kNN) queries, a computationally expensive approach that produces poorly aligned spatial receptive fields. We can add that in practice, kNN queries and relative positional encoding operations represent important computational bottlenecks, consuming approximately 22.2% and 7.8% of the total forward propagation time, respectively (see Figure 12).

7. Beyond Point Transformer v1

7.1. PTv2: Grouped Attention

To address the architectural bottlenecks of its predecessor and make the model more practical for large-scale point cloud tasks, Point Transformer V2 [14] revises the attention, position encoding and pooling mechanisms, as illustrated in Figure 13. Specifically, it introduces Grouped Vector Attention (GVA), which partitions the value vector into groups that share attention weights. This mechanism drastically reduces the number of parameters of the weight encoding layer and improves computational efficiency and generalization compared to the original vector attention mechanism. Furthermore, the authors strengthen the spatial reasoning by adding a position encoding multiplier to better exploit the continuous 3D relationships through geometry-dependent amplification or attenuation.

Instead of relying solely on an additive position bias like in the first version, the attention weights are now formulated as $w_{ij} = \omega(\delta_{mul}(\Delta p_{ij}) \odot \gamma(q_i, k_j) + \delta_{bias}(\Delta p_{ij}))$ where Δp_{ij} is the relative position between two points, γ is the relation vector between the query and key, $\odot$ represents the Hadamard product, and both δ_{mul} and δ_{bias} are learnable MLPs.

For downsampling, the model replaces the traditional sampling-and-query pooling pipeline (FPS combined with kNN) with a lightweight partition-based pooling strategy that divides the space into non-overlapping uniform grids, improving the spatial alignment and computational efficiency. Regarding performance, these improvements allow Point Transformer V2 to establish a better performance on the ModelNet40 shape classification benchmark, reaching an overall accuracy of 94.2% and a mean class accuracy of 91.6%, improving upon the 93.7% overall accuracy reported for the first version.

7.2. PTv3: Sequence Modeling

Point Transformer v3 [13] pushes this evolution even further and marks a paradigm shift, prioritizing large-scale efficiency over complex attention mechanisms in order to overcome the accuracy-efficiency trade-off. The intuition here is particularly interesting: given that transformers natively rely on 1D sequences, the authors bypass the massive computational bottleneck of kNN searches, which were still strictly relied upon until now to preserve permutation invariance. They achieve this by directly translating the unstructured 3D data into a standard sequential format. As illustrated in Figure 14, the point clouds are serialized using space-filling curves, such as the Z-order / Hilbert curves, which transform the 3D model into an ordered sequence while preserving the notion of spatial neighborhoods as much as possible. The sequence is then sliced into patches and projected into descriptors, completely replacing memory-intensive neighborhood attention with highly efficient windowed attention. As one might deduce from

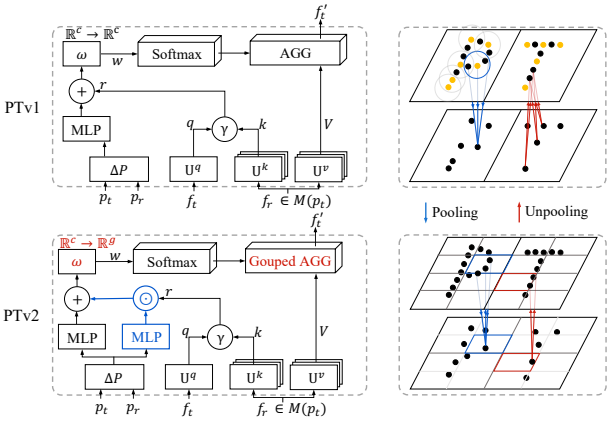

Figure 13. Comparison of the attention, position encoding, and pooling mechanisms between PTv1 and PTv2. Top-left: the original vector attention with position encoding in PTv1. Bottom-left: the proposed grouped vector attention (indicated in red) with improved position encoding (indicated in blue) in PTv2. Top-right: the sampling-based pooling and interpolation-based unpooling in PTv1. Bottom-right: the proposed partition-based pooling and unpooling in PTv2. (Source: [14])

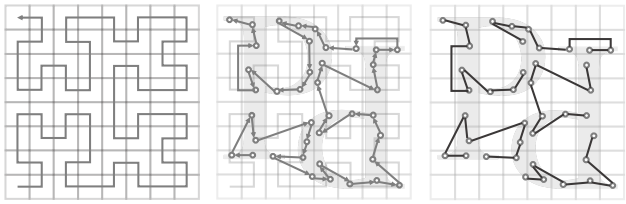

Figure 14. Hilbert point cloud serialization. The space-filling curve for serialization (left), point cloud serialization var sorting order within the space-filling curve (middle), and grouped patches of the serialized point cloud for local attention (right). (Source: [13])

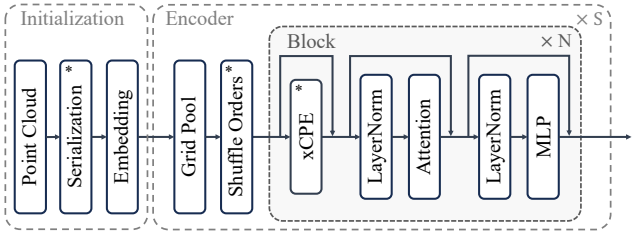

Figure 15. Overall architecture of PTv3 (source: [13]).

this mapping strategy, translating a 3D space to a 1D sequence occasionally places spatially close points into different patches; however, the model's shift interactions easily resolve these boundary issues. Furthermore, the computationally heavy relative positional encoding is completely removed and replaced by a simple prepositive sparse convolutional layer, acting as an efficient Conditional Positional Encoding (xCPE) (see Figure 15). These radical simplifications result in a network that is 3.3 times faster and consumes 10.2 times less memory than version 2, all while expanding the receptive field from 16 to 1024 points. It is important to note that Point Transformer V3, focusing entirely on large-scale, scene-centric perception tasks, does not report performance metrics for the ModelNet40 classification benchmark. Thus, a direct performance comparison cannot be made here.

8. Conclusion and Further Work

The paper presents an adaptation of Transformers and self-attention to 3D point cloud processing through a specialized local vector self-attention mechanism. In this project, my primary focus was adapting the segmentation-centric framework to reproduce and evaluate their 3D shape classification results on the ModelNet40 dataset. My best model obtained an overall accuracy of 92.7% on the test dataset, which falls slightly short of the 93.7% reported by the authors. However, I believe that if I trained the model for 200 epochs instead of 100, I would probably reach their reported accuracy. Furthermore, with more extensive hyperparameter tuning, it should be possible to improve my best model and close the remaining gap between my implementation and theirs.

Beyond the baseline reproduction, I carried out several ablation studies, some of which did not appear in the original paper. These studies revealed that the model's success heavily relies on the synergy between a restricted local neighborhood size, relative positional encoding, vector-based attention, and standard SGD optimization. To share these findings and my implementation for shape classification on ModelNet40, I plan to open a pull request to the original repository in the near future.

For future work, a logical next step would be to implement and evaluate the newer architectures (PTv2 and PTv3) on the ModelNet40 classification task to directly quantify their training speed and accuracy gains compared to the baseline v1. Additionally, testing the current implementation on dense, large-scale prediction datasets, such as S3DIS [1] or ScanNet [3], would provide a better understanding of how the vector attention mechanism adapts to highly complex semantic scenes.

References

[1] Iro Armeni, Savva Sax, Amir R. Zamir, and Silvio Savarese. 3d semantic parsing of large-scale indoor spaces. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 1, 10

[2] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository, 2015. 1

[3] Angela Dai, Angel X. Chang, Manolis Savva, Maciej Halber, Thomas Funkhouser, and Matthias Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes, 2017. 10

[4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. 1

[5] Niki Parmar, Ashish Vaswani, Jakob Uszkoreit, Łukasz Kaiser, Noam Shazeer, Alexander Ku, and Dustin Tran. Image transformer, 2018. 1

[6] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation, 2017. 2

[7] Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, 2017. 2, 6

[8] Xavier Roynard, Jean-Emmanuel Deschaud, and François Goulette. Classification of point cloud scenes with multiscale voxel deep network, 2018. 1

[9] Hugues Thomas, Charles R. Qi, Jean-Emmanuel Deschaud, Beatriz Marcotegui, François Goulette, and Leonidas J. Guibas. Kpconv: Flexible and deformable convolution for point clouds, 2019. 2

[10] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers, 2021. 1

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. 3

[12] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. Dynamic graph cnn for learning on point clouds, 2019. 2

[13] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer v3: Simpler, faster, stronger, 2024. 9

[14] Xiaoyang Wu, Yixing Lao, Li Jiang, Xihui Liu, and Hengshuang Zhao. Point transformer v2: Grouped vector attention and partition-based pooling, 2022. 9

[15] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes, 2015. 1, 5

[16] Hengshuang Zhao, Jiaya Jia, and Vladlen Koltun. Exploring self-attention for image recognition, 2020. 1, 3

[17] Hengshuang Zhao, Li Jiang, Jiaya Jia, Philip Torr, and Vladlen Koltun. Point transformer, 2021. 1, 3, 4

[18] Yin Zhou and Oncel Tuzel. Voxelnet: End-to-end learning for point cloud based 3d object detection, 2017. 1